Two-Tower Cross-Attention BiGRU with Passage Alignment for Multi-Sentence Reading Comprehension

Group 34

1 Introduction

For this project, we focus on the Multi-Sentence Reading Comprehension (MultiRC) benchmark (Khashabi et al., 2018), developing a Question Answering (QA) model trained strictly on the dataset. MultiRC evaluates reading comprehension beyond the simple string-span identification of traditional QA benchmarks (Rajpurkar et al., 2016), requiring models to use information across multiple sentences to answer questions. Furthermore, because each question can have multiple correct answers, it challenges the simple approach of applying a softmax to select a single most confident answer. The multi-sentence synthesis and 'reasoning' that this task requires is far more useful for real-world NLP applications, such as clinical note review or legal contract analysis. Our aim is to tackle these challenges and test the limits of performance when training strictly from scratch without external data.

To address this, we design, implement, and comprehensively evaluate a modular Two-Tower Bidirectional Gated Recurrent Unit (BiGRU) (Cho et al., 2014) trained from scratch for multi-sentence reasoning. Our architecture links separated passage and question–answer representations with multi-head cross-attention, compresses sequence states with attention pooling, and incorporates explicit alignment features into the classification head. To handle MultiRC's multi-answer structure and passage difficulties (where complex passages often yield low overall logit confidence), we train with a custom Within-Question loss centring candidate scores around the question mean, applying a ranking penalty between valid and invalid answers.

We evaluate our model against eight baselines across four tiers: trivial heuristics, static embeddings, frozen pre-trained BiGRUs, and a zero-shot LLM ceiling. In our 37-variation ablation study across three random seeds, our reference architecture achieves a test within-question ranking AUC (qAUC) of 0.686, outperforming static embedding similarity baselines (0.650) and approaching recurrent models with pre-trained FastText embeddings (0.714). Our ablations showed that attention mechanisms offer only minor improvements, while explicit alignment features in the classifier head provide the primary reasoning signal ($\Delta = -0.043$ when removed), suggesting models trained from scratch on small datasets rely heavily on direct token matching rather than multi-sentence reasoning.

2 Methodology

2.1 Architecture Overview and Diagram

Figure 1 illustrates our reference Two-Tower BiGRU (`ix+wq`) architecture. To prevent long passages from drowning out short candidate answers, we decouple the input into separate Passage and QA towers that share a token embedding layer ($d = 128$) before passing through dedicated BiGRU encoders.

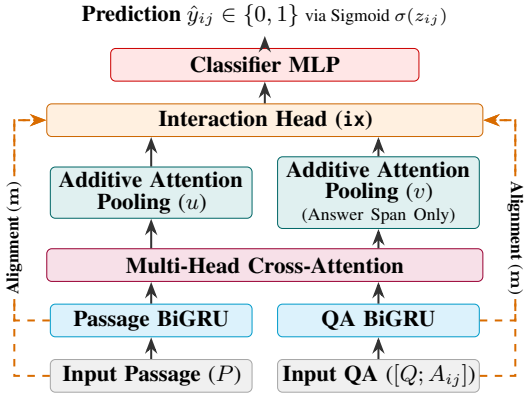


Figure 1: Two-Tower Cross-Attention Reference Architecture (`ix+wq`). The dashed bypass feeds pairwise Passage–QA alignment features ($\mathbf{m} \in \mathbb{R}^6$) directly into the Interaction Head.

2.2 Model Components and Attention Mechanism

RNN Architecture. We use a BiGRU as our sequential backbone. Bidirectional processing gives every token context from both sides, which multi-sentence reasoning requires. With only 456 training passages and 17,343 unique words, larger architectures overfit badly: our initial small transformer fit the training set well, but generalised poorly on test data. A GRU needs only reset and update gates per step against four for an LSTM, giving a higher ceiling than a plain RNN at a manageable parameter cost. RNNs also handle sequence order without the positional embeddings that a transformer requires, and with so little data our from-scratch embeddings failed to learn useful generalisations, leading us back to the recurrent design of the original MultiRC paper.

Attention and Pooling. Our reference model applies multi-head cross-attention (Vaswani et al., 2017) so each tower can retrieve context from the other, taking the QA tower as query Q and the passage as keys/values K, V:

$$\text{Cross}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M_p\right)V \quad (1)$$

where M_p masks padding <pad> tokens, $H = 4$ attention heads, and $d_k = d/H = 32$. Passages also attend symmetrically to the QA tower in the other direction.

To compress variable-length sequences into fixed vectors while filtering unimportant tokens we use feedforward additive attention pooling (Bahdanau et al., 2015), scored by $e_t = w^T \tanh(W_a h_t + b_a)$ and normalised as $\alpha_t = \exp(e_t)/\sum_k \exp(e_k)$, which is pooled over the whole passage. In the QA tower, pooling is restricted to the tokens of the candidate answer, so generic question tokens don't add noise.

Alignment Features ($\mathbf{m}$). To limit compression loss over multi-sentence passages we also compute a 6-dimensional alignment vector $\mathbf{m}$, concatenated onto the classifier input: m_1, m_2 are the exact precision and recall of candidate answer tokens within the passage;m_3, m_4 are the maximum and mean cosine similarity between answer and passage token embeddings (via a learned projection W_m initialised to I); and m_5, m_6 are the maximum and mean scaled dot-product similarity between $\tanh$-projected QA and Passage BiGRU hidden states.

Interaction Head and Training Objective. The pooled representations and alignment features are concatenated with the interaction representation $h_{\text{clf}} = [u;\, v;\, |u-v|;\, u \odot v;\, \mathbf{m}]$. A two-layer MLP with dropout ($p = 0.2$) computes a logit $z_{ij} = W_2 \text{ReLU}(W_1 h_{\text{clf}} + b_1) + b_2$.

To avoid passage difficulty from lowering prediction confidence and affecting training, we use mean centring per question ($\tilde{z}_{ij} = z_{ij} - \bar{z}_i$, where $\bar{z}_i = \frac{1}{C_i}\sum_{k=1}^{C_i} z_{ik}$) and train using our Within-Question (wq) loss function:

$$\mathcal{L}_{\text{wq}} = \frac{1}{C_i}\sum_j \text{BCE}(\sigma(\tilde{z}_{ij}), y_{ij}) + \frac{\alpha}{|\mathcal{P}_i|}\sum_{(j,k)\in\mathcal{P}_i} \text{softplus}(\tilde{z}_{ik} - \tilde{z}_{ij}) \quad (2)$$

where $\mathcal{P}_i = \{(j,k) \mid y_{ij} = 1, y_{ik} = 0\}$ indexes candidate pairs with opposing labels and $\alpha = 0.5$.

3 Experimental Setup

3.1 Dataset Description

MultiRC has the largest flattened size of the available datasets (27,243 training and 4,848 test, against 4,638 for ReClor (Yu et al., 2020) and roughly 16,000 for PIQA (Bisk et al., 2020)). Because we are training from scratch, this matters for reducing overfitting, and the fact that more than one answer may be correct posed an interesting challenge for us.

MultiRC is organised into a three-tier structure; paragraphs, questions, and candidate answers. Each paragraph is a multi-sentence passage (∼10-15 sentences, averaging 300 words) from domains such as news, fiction, and encyclopedias, with a mean of 11 questions per passage and 5 candidate answers per question. Supporting sentence indices (`sentences_used`) and a multi-sentence reasoning flag (`multisent`, evaluation only) are also provided. Unlike single-best-option tasks, each candidate is judged independently, so any one or more may be correct.

During preprocessing we strip HTML sentence markers and join sentences with <sep> tokens to keep boundaries visible to the encoder, then flatten the hierarchy into per-candidate tuples (P_i, Q_i, A_{ij}). The 456 training passages are partitioned 82%/18% at the passage level into 374 training and 82 validation passages (22,926 and 4,317 tuples respectively), so no passage appears in both; the provided 83-passage split is held out for testing. Candidates sharing a (P, Q) pair share a question ID, keeping them together. Positive answers account for 44.10% of training and 42.80% of test.

To prevent sequence truncation from discarding critical evidence we employ a head/tail passage truncation strategy. Long passages with $L_p \geq 448$ tokens retain the first and last 224 tokens, preserving supporting facts for 96.7% of truncated instances (compared to just 62.3% for head-only truncation). Questions and candidate answers are capped at $L_q = 32$ and $L_a = 28$ tokens respectively. We tokenise all text into lowercase words, dropping punctuation to save ∼13% of our sequence length budget without degrading performance. We made the decision to retain stopwords, as removing them degraded token alignment features by -0.0155 qAUC. Vocabulary is constructed strictly from the training split. Due to the limited size, we constructed with min_freq = 1, resulting in 17,343 tokens (with a test data OOV rate of 7.8%).

3.2 Training Procedure and Hyperparameters

To train our models, we used the AdamW optimizer (Loshchilov and Hutter, 2019) with a base learning rate of $\eta = 3 \times 10^{-4}$ and gradient clipping capped at a maximum norm of 1.0 to guard against exploding gradients. To avoid destabilising the model while weights are still random, we used a 2-epoch linear warmup, after which we applied a cosine decay schedule over 12 epochs to a floor of 1×10^{-5}. We chose a 12-epoch ceiling to allow the cosine schedule to actually anneal before early stopping consistently triggered around epoch 7.

To prevent overfitting, we applied a dropout rate of $p = 0.2$ across embeddings, attention layers, and classifier projections, as well as a weight decay of $\lambda = 0.01$ applied to weight matrices (not biases

or embeddings). Mini-batches were maintained at size $B = 64$ using our `GroupedBatchSampler` to ensure all candidate options for a question stayed together, and models were trained using our `wq` loss with pairwise ranking penalty $\alpha = 0.5$.

For model selection, we used early stopping with a patience of 5 epochs, a minimum improvement delta of $\Delta = 0.001$, and minimum of 4 epochs, tracking validation qAUC. We selected checkpoints based on qAUC rather than per-question Macro $F_1 (qF_1)$ (which jittered by ± 0.03–0.08 per epoch), as qAUC is threshold-independent, varied by only ~ 0.004, and directly rewards the model for ranking correct answers above incorrect ones. Setting $\Delta = 0.001$ ensured our stopping criterion sat safely below this noise floor.

3.3 Baseline Models

Our "default" model choice used during implementation was a Two-Tower architecture with Cross-Attention, Interaction Head (`ix`) classifier input and `wq` loss function (`two_tower/cross/attention/ix+wq`). This was compared against 8 baseline models:

- **Always Yes/No.** With a class imbalance of 44.80% "yes" answers these are the simplest baselines, and immediately identify when a model collapses into an "always-no" state.
- **Lexical Overlap.** Ignores the question and scores the maximum proportion of answer tokens appearing in any single passage sentence.
- **Static Embeddings (GloVe / FastText).** Maximum cosine similarity between pooled sentence vectors and the QA vector, using pre-trained 300-d GloVe (Pennington et al., 2014) or FastText (Bojanowski et al., 2017) embeddings.
- **Frozen BiGRU Baselines.** The same Two-Tower approach as our main model, but with pre-trained GloVe or FastText embeddings, enabling deeper semantics unconstrained by training data size.
- **FLAN-T5 Base** (Chung et al., 2024). As an upper ceiling, an instruction-tuned 250M-parameter LLM answering a zero-shot yes/no prompt ("Passage: ... Question: ... Answer: ... Is the answer supported by the passage? Answer yes or no."). We softmax the logits of the "yes"/"no" output tokens for a confidence score rather than accepting the response string.

3.4 Evaluation Metrics

To capture both isolated decision quality and complete multi-sentence reasoning, we evaluate models across two distinct levels: global candidate-level classification, and the canonical per-question evaluation established by Khashabi et al. (2018). Candidate-level metrics evaluate individual passage–question–answer triples, while per-question evaluations involve all candidates related to a particular question.

Question-Level Metrics. We measure the canonical MultiRC benchmarks as follows:

- **Exact Match (qEM):** Scores 1.0 iff all C_i candidate answers for question i are predicted correctly: $\text{qEM} = \frac{1}{N_q} \sum_{i=1}^{N_q} \prod_{j=1}^{C_i} \mathbb{I}(\hat{y}_{ij} = y_{ij})$. Because any single error results in a score of 0, it is impossible to "game" (our "always-yes" baseline scores just 0.0084).
- **Per-Question Macro F_1 (qF_1):** Computes the harmonic mean of precision and recall for each question, averaged across all questions: $qF_1 = \frac{1}{N_q} \sum_{i=1}^{N_q} F_1(y_i, \hat{y}_i)$. We use qF_1 because unlike global F_1, it weights every question equally regardless of candidate count, providing useful information with small and large questions alike.
- **Within-Question ROC-AUC (qAUC):** the probability that a random positive answer outranks a random negative one for the same question. Because it is threshold-independent, we choose it as our checkpoint-selection metric, so early stopping is not fired prematurely on models whose confidences sit below the standard "yes" threshold of 0.5.

Decision Threshold Tuning. Models often rank "yes" answers above "no" answers while scoring both below the default 0.5 cut. Sweeping thresholds to maximise answer-level F_1 selected a low boundary ($t = 0.33$), predicting positive for 84.0% of candidates and buying recall at the expense of precision. We therefore use Prior-Matching Calibration: scores are centred within each question, $\tilde{s}_{ij} = s_{ij} - \frac{1}{C_i} \sum_k s_{ik}$, and the threshold is set to reproduce the dataset's positive prevalence (44.41%) on the validation predictions to guarantee optimised cutoff thresholds without gaming the metric.

4 Results and Analysis

4.1 Overall Performance Comparison

Comparison Against Baselines. Table 1 and Figure 2 compare our reference model against selected baselines. Our reference `Two-Tower BiGRU (ix+wq)` architecture achieves 0.6855 test within-question AUC (qAUC) and 14.62% question exact match (qEM), significantly surpassing heuristic always-yes/no baselines (0.5000 qAUC). Importantly, while (`Always-Yes`) achieves a test qF_1 of 59.10%, its exact match collapses to 0.84% (qEM = 0.0084), highlighting the importance of the qAUC metric. As illustrated by the distribution in Figure 2, our model shows clear separation from simple Jaccard lexical overlap (0.656 qAUC) and

Table 1: Test evaluation comparing our Main Model (`two_tower/cross/attention/ix+wq`) against baseline benchmarks.

Model / System	Val qAUC	Test qAUC	Test qEM	Test qF_1
Tier 1: Trivial & Heuristic Baselines				
Always-No	0.5000	0.5000	0.31%	0.31%
Always-Yes	0.5000	0.5000	0.84%	59.10%
Lexical Overlap	0.6636	0.6565	13.01%	51.35%
Tier 2: Static Embedding Similarity (Frozen)				
FastText (Cosine)	0.5852	0.6387	12.28%	52.73%
GloVe (Cosine)	0.6194	0.6502	15.32%	53.84%
Tier 3: Supervised BiGRU Baselines (Frozen)				
GloVe + BiGRU	0.7402	0.6938	14.59%	53.34%
FastText + BiGRU	**0.7583**	**0.7144**	**15.53%**	**55.69%**
Our Main Model (Trained from Scratch)				
Two-Tower BiGRU (`ix+wq`)	0.7300	0.6855	14.62%	54.32%
Tier 4: Zero-Shot Pretrained LLM (Ceiling)				
FLAN-T5 (Base)	*0.8923*	*0.8695*	*37.15%*	*72.48%*

pre-trained vector cosine similarities (FastText at 0.6387, GloVe at 0.6502), confirming that learned sequence representations are important in multi-sentence question answering.

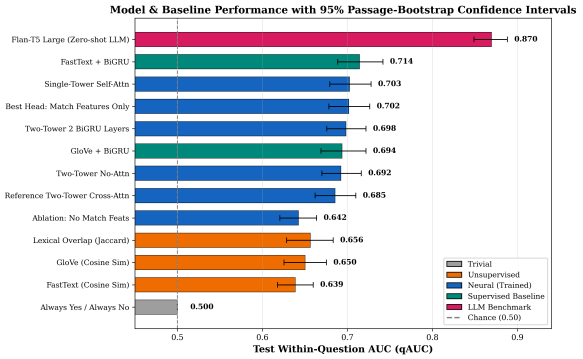


Figure 2: Baseline Performance Comparison

Recurrent Baselines and the Pre-trained Ceiling. Compared to recurrent models trained with frozen 300-dimensional pre-trained vectors, our reference model performs competitively with `GloVe+BiGRU` (0.6938 qAUC, 14.59% qEM), but slightly below `FastText+BiGRU` (0.7144 qAUC, 15.53% qEM, 55.69% qF_1), due to the slightly higher vocabulary coverage of FastText with its sub-word encoding. However, our 95% confidence intervals in Figure 2 show that the reference model's CI [0.6620, 0.7100] overlaps with both `GloVe+BiGRU` [0.6688, 0.7219] and `FastText+BiGRU` [0.6887, 0.7419], meaning this gap is not statistically significant at $p < 0.05$. Finally, the `FLAN-T5` benchmark (0.8695 test qAUC, 37.15% qEM, 95% CI [0.8488, 0.8882]) shows the architectural performance ceiling of our RNNs, both from-scratch and pretrained, compared to pretrained Transformer models.

4.2 Ablation Studies

Additional Components. For our detailed ablation study, we included several other changeable components in our model:

- **Passage Recurrent Modelling:** To assess the impact of sequential modelling, we ablate the passage BiGRU by bypassing it entirely and feeding static word embeddings directly into the cross-attention layer.
- **Encoder Depth and Hidden Capacity (`num_layers_2`, `d_model_256`):** We test stacking two BiGRU layers (`num_layers = 2`, inter-layer dropout $p = 0.2$) and doubling the hidden dimension from $d = 128$ to $d = 256$. Increasing parameter size allows us to test how prone our model is to overfitting.
- **Attention Pooling Functions (`mean`, `max`, `final`):** We compare our baseline Additive Attention Pooling against 3 other pooling methods to see how different approaches to compressing variable-length passages into a fixed-length embedding affect performance.
- **Classifier Input Formulations (`head_concat`, `head_compare`, `head_match`):** Given pooled passage vector u and QA vector v, we compare how representations are passed to the final classifier:
  - **Interaction Head (`ix`):** $h_{\text{clf}} = [u; v; |u - v|; u \odot v]$, supplying raw representations, absolute difference, and element-wise product.
  - **Concatenation Head (`cc`):** $h_{\text{clf}} = [u; v]$, forcing the classifier to discover relational interactions on its own.
  - **Comparison Head (`compare`):** $h_{\text{clf}} = [|u - v|; u \odot v]$, restricting the classifier strictly to relational distance.
  - **Match Head (`match`):** Bypasses pooled vectors entirely, passing only the alignment features m to the classifier.
- **Within-Question Loss vs. BCE (`loss_bce`, `loss_no_pairwise`):** Standard Binary Cross-Entropy (`loss_bce`) evaluates each passage–question–answer triple in isolation, ignoring passage difficulty. Complex passages often yield lower logits even when correct answers outrank

Table 2: Controlled ablation study (3-seed mean). Δ denotes the test qAUC difference relative to the Reference (0.685); qEM and qF1 are the metrics at the declared decision threshold.

Component Variant	Val qAUC	Test qAUC	Δ	Test qEM	Test qF1
Reference Architecture					
Two-Tower Cross-Attn [Ref]	0.730	0.685	–	0.146	0.543
Attention Mechanism					
Inter-Attention (token match)	0.724	0.680	-0.006	0.148	0.554
Self-Attention	0.732	0.688	+0.002	0.149	0.549
No Attention	0.737	0.692	+0.007	**0.159**	0.553
Single-Tower Self-Attn	0.733	**0.703**	**+0.017**	0.155	0.533
Pooling Method					
Mean Pooling	0.729	0.687	+0.001	0.155	0.563
Max Pooling	0.735	0.694	+0.008	0.156	0.559
Final State Only	0.726	0.684	-0.002	0.153	0.564
Classifier Head					
Match features only (no pooled)	**0.741**	0.702	+0.016	0.156	0.557
Concat $[u; v]$ only	0.730	0.689	+0.004	0.147	**0.569**
Compare $[\|u - v\|; u \odot v]$ only	0.731	0.688	+0.002	0.156	0.554
Interaction w/o match feats	0.660	0.642	-0.043	0.134	0.543
Loss Objective					
Standard BCE (no ranking)	0.724	0.684	-0.001	0.150	0.559
No pairwise margin ($\alpha = 0$)	0.734	0.688	+0.003	0.143	0.555
Capacity & Regularization					
Passage w/o BiGRU	0.732	0.687	+0.002	0.154	0.559
Hidden size $d_{\text{model}} = 256$	0.726	0.691	+0.006	0.147	0.559
2 BiGRU layers	0.733	0.698	+0.013	**0.159**	0.562
Dropout $p = 0.4$	0.720	0.681	-0.005	0.139	0.562

incorrect ones, causing BCE to penalise valid predictions. For this, our Within-Question loss centres logits around the question mean before computing BCE, and adds a softplus pairwise ranking penalty scaled by error magnitude (2).

To examine each component's contribution, we ran controlled ablations off our reference model ([ix+wq]). We evaluated 37 single-component ablations and architectural permutations across three random seeds ($n = 3$), holding all non-ablated components, hyperparameters, and early stopping criteria strictly fixed relative to our reference model. Table 2 reports validation and test metrics alongside test ranking deltas (Δ) relative to the reference model (full results are in our submitted notebook).

Role of Alignment Features and Classifier Head. Our most critical finding came from ablating the classifier inputs. Removing passage alignment features (`Interaction w/o match feats`) caused the largest performance drop ($\Delta = -0.043$ qAUC, from 0.685 to 0.642; exact match $0.146 \rightarrow 0.134$). This confirms our model relies on matching keywords directly in the passage, aligning with our hypothesis (motivated by techniques like Named Entity Recognition) that explicit alignment features would outperform dense embeddings alone.

Conversely, restricting the classifier exclusively to alignment features (`head_match`, discarding $[u; v]$) yields our strongest validation ranking (Val qAUC $= 0.741$) and a strong test ranking (qAUC $= 0.702$, $\Delta = +0.016$). Removing pooled representations physically prevents the model from memorising question-answer correlations that fail to generalise to unseen passages. However, relying purely on keyword presence/absence rather than multi-sentence reasoning also explains why the model still has a limited performance ceiling.

Attention. Surprisingly, cross-attention (0.686 test qAUC) and inter-attention (0.680) offer no advantage over self-attention, which in turn offers no advantage over removing attention entirely (`No-Attn` reaches 0.692). This differed from our hypothesis that cross-attention between the passage and QA pair would outperform standard attention (or at least beat no attention at all!). Instead, this further indicates that the model relies on hard alignment features and token matching rather than multi-sentence reasoning. A detailed qualitative analysis of our attention mechanisms follows below.

Single-Tower vs. Two-Tower. Our single-tower architecture performs on par with or slightly better than the two-tower variants (best single-tower `single/none/mean/ix+wq` reaches 0.703 test qAUC vs. 0.685 for our reference). Separating passage and QA sequences provides no real benefit, suggesting the single tower relates the two on its own like an internal cross-attention mechanism.

Pooling Strategies. Across both tower configurations, `max` (0.694 test qAUC) and `mean` (0.687) pooling consistently outperform `attention` (0.685) and `final state` (0.684) pooling. Max-pooling is well-suited for MultiRC because it preserves peak activation signals from the specific evidence sentence regardless of where it appears, whereas `mean` and `final` pooling are more easily diluted by filler or red-herring statements.

Loss Objectives. Our Within-Question ranking loss (`ix+wq`) achieves a similar validation score (0.730 Val qAUC) to standard BCE (0.724) and mean-centring alone (0.734). This suggests that logit magnitude variations between easy and hard passages did not meaningfully impact training.

Model Capacity and Regularization. Stacking 2 BiGRU layers improves test qAUC to 0.698 and achieves our highest test qEM (0.159). Conversely, increasing dropout from 0.2 to 0.4 reduces performance (0.681 test qAUC, 0.139 qEM), indicating that severe overfitting was not occurring. This surprised us, as we suspected our models were drastically overfitting, but it appears early stopping functioned properly and we were simply approaching our architecture's representation ceiling.

Importantly, removing the passage BiGRU entirely (`passage_no_gru`) still achieves 0.687 test qAUC. This confirms that our models rely almost entirely on static embeddings and alignment features to match tokens directly, rather than performing sequential multi-sentence reasoning.

Reference Model Justification. Overall, while our ablations supported our decision to include explicit alignment features (m) and the interaction head (`ix`), they didn't prove our initial assumption that complex cross-attention and two-tower separation would perform better than simple pooling (`max`) in a single-tower architecture. We selected this reference architecture to follow a hypothesis-driven approach to our experiments, as this was the model we expected to have the greatest performance, and as such we positioned our ablation study to isolate the impacts of individual components compared to the reference. Despite not achieving the highest results, we feel that the intuitive interpretability offered by the reference architecture offers valuable insight into the operation of the model, something not offered as deeply in a single tower architecture with no attention. As such, we believe that our reference model has earned its place as our most informative architecture.

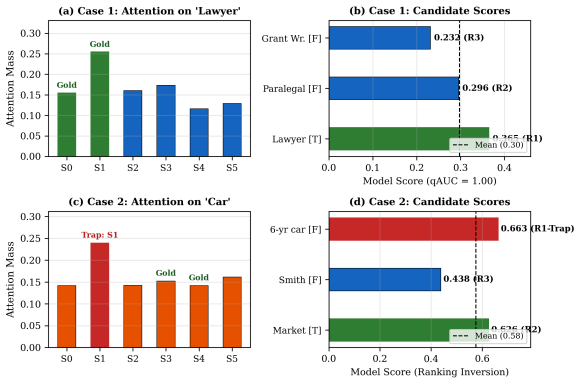


Figure 3: Sentence Level Attention

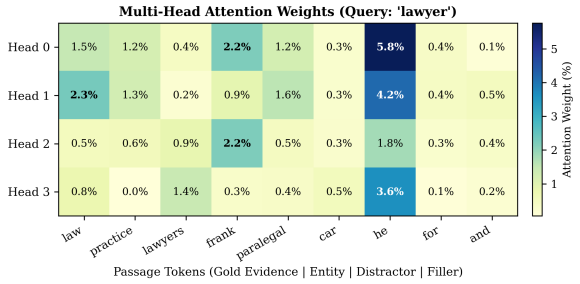


Figure 4: Attention Heads

4.3 Qualitative Analysis of Attention

To investigate how attention and pooling mechanisms interact, we observed individual sentence-level attentions from our trained reference model on test passages, as illustrated in Figure 3.

In Case 1 (Figure 3a), the model addresses the question *"What is Frank Smith's profession?"* with options *"Lawyer"* (true), *"Paralegal"* (false) and *"Grant Writer"* (false). Supporting evidence is split across sentence S0 (*"lucrative law practice"*) and sentence S1 (*"paying his senior paralegal minimum wage"*). When querying the candidate answer `Lawyer`, cross-attention focuses strongly on both S0 and S1 ($> 52\%$ of total attention). This evidence grounding successfully avoids the distractor `Paralegal` (0.296) and makes `Lawyer` the highest ranked answer (0.365, achieving perfect qAUC $= 1.00$ and exact match). This shows that our attention can successfully use multi-sentence context to answer questions correctly.

Multi-Head Attention. Highlighted in (Figure 4), we can see a breakdown of what the model attended to for the specific sequence tokens *"[what, is, frank, smith, s, profession, <sep>, lawyer, <sep>]"*. With query *"lawyer"*, the model specifically attended to important terms *"he"*, *"frank"* and *"law"*, while stop-words *"for"* and *"and"* received very low attention. Importantly, Head 0 focused heavily on entities *"he"* and *"frank"*, Head 1 attended to *"law"*, *"practice"*, while all four heads suppressed the distractor noun *"car"*. This shows the specialisation of individual heads in multi-head attention, each head sensitive to different areas due to the unique random initialisations during training.

The Lexical Overlap Shortcut. Unfortunately, Case 2 (Figure 3b) highlights the main problem with our model, and likely the reason no models were able to raise the performance ceiling. For the comparative question *"Who charges more for services: Frank Smith, or the lawyer's market in general?"*, a model must compare evidence in S3 ($50/hr minimum) against evidence in S4 ($150–$250/hr market rate).

While the correct answer `The lawyer's market` had high confidence (0.626), the distractor `The 6-year-old car` was incorrectly ranked highest (0.663). We suspect that because sentence S1 literally contains the phrase *"Frank Smith drives a 6-year-old car"*, the model focuses on this sentence due to the presence of the lexical-matched *"Frank Smith"*, rather than the evidence sentences S3 and S4. Combined with high explicit match features, the model predicts the automobile as an entity.

This error validates our quantitative ablation findings: with limited training data lacking detailed real-world knowledge, our models are relying almost entirely upon strict token matching between questions and passages, rather than performing actual multi-sentence reasoning.

5 Conclusion

Our design, evaluation, and critical ablation of recurrent sequence architectures for multi-sentence reading comprehension on the MultiRC dataset found that a Two-Tower BiGRU trained from scratch outperforms trivial heuristics and pretrained embedding vector similarities, and performs similarly to supervised baselines initialised with pretrained 300-dimensional embeddings.

We show that specific feature matching accounts for the vast majority of model performance, while cross/self-attention mechanisms achieved minimal performance uplift over simple pooling or single-tower encoders, likely due to attention noise across long passages. While our architectures achieve strong within-question performance (test qAUC of ≈ 0.69–0.70), the Question Exact Match metric remains bounded near 15%. The significant performance margin of the FLAN-T5 benchmark highlights how multi-sentence reasoning is difficult to learn in small, from-scratch recurrent models.

Future improvements may include investigating sub-word tokenisers to increase vocabulary coverage, testing LSTM or alternative RNN architectures, or utilising data augmentation techniques such as entity swapping to mitigate the lexical matching shortcuts.

Team Contributions

During the research phase of our project, both of us agreed upon our reference architecture, and some of the components we would want to test on it. All of our code was written in a shared Molab python notebook (far superior to Google Colab, we got to train on an RTX Pro 6000!) , and our report was written together in a shared Overleaf project.

Tobias: I was responsible for the initial notebook setup and core architecture implementation, including the Two-Tower BiGRU encoders, multi-head cross-attention mechanism, and attention pooling. I came up with the Within-Question ranking loss (wq) after BCE loss seemed to have training issues (little did I know it wouldn't make any real difference), set up the multi-seed ablation experiment harness, implemented the Prior-Matching threshold calibration and drafted the Methodology and Model Component sections. Researching these components took a long time, and their implementation even longer, and longer still fighting with models that refused to train past the first epoch.

Etienne: I worked on the initial dataset analysis and engineered the preprocessing and batching pipeline. This included the train/validation splitting, truncation strategy, and vocabulary construction. I implemented most of the evaluation strategies after discussion with Tobias. I also researched and selected our baseline model benchmarks based on what we discovered during initial training, and from my interest at the start of testing a lightweight LLM (FLAN-T5) on the task. Alongside this I kicked off the report with the structure, diagram, and of course working through explaining our system within the tight length limit.

As development progressed (and our models didn't get any better), we each added more components that we were interested in, until we were ready to run our ablation grid and write the report. Overall, our distribution of work was well allocated, and we worked exceptionally well as a team :)

Generative AI Disclaimer

We used Gemini 3.8 Flash and Deepseek v4.1 Flash as a coding assistant during the development of this project, which were almost entirely limited to creating matplotlib graphs and output logging, a decision we felt was justified by the boilerplate nature of this code. All architectural decisions were 100% our own, made either from course content or from our own research (which involved both traditional Google searches and the built-in AI results overview). All sections of this report were handwritten, with minor tweaks from the Overleaf AI Assistant used strictly to help with the complex LaTeX formatting, and reworking our diagram from draw.io to LaTeX's `tikz`, something that we were relatively unfamiliar with.

Additionally, we used a coding assistant to calculate the 95% confidence intervals as part of (Figure 2) and the specific syntactic implementation of the feedforward additive attention pooling in line with the original paper by Bahdanau et al. (2015).

References

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In *3rd International Conference on Learning Representations (ICLR)*.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 34, pages 7432–7439.

Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics (TACL)*, 5:135–146.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar. Association for Computational Linguistics.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research (JMLR)*, 25(70):1–53.

Daniel Khashabi, Snigdha Chaturvedi, Michael Roth, Shyam Upadhyay, and Dan Roth. 2018. Looking beyond the surface: A challenge to address true multi-hop reading comprehension. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 252–262.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*.

Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2383–2392.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.

Weihao Yu, Zihang Jiang, Yanfei Dong, and Jiashi Feng. 2020. ReClor: A reading comprehension dataset requiring logical reasoning. In *International Conference on Learning Representations (ICLR)*.